

Giornata 3 — ETL e Data Warehouse

SAC DataFlow × MediStore | Documento di consegna ufficiale per il team.

Brief del CTO

Bene. Il database funziona, le query rispondono.
Ma adesso immaginate che tra un mese MediStore ci manda un nuovo CSV.
Cosa fate? Ripulite tutto a mano? No.
Dovete costruire una pipeline che prenda i dati grezzi, li pulisca,
li trasformi e li carichi nel Data Warehouse in modo automatico e ripetibile.
Nota: non abbiamo fatto il connettore Python-MySQL.
Lo script Python produrrà quindi i CSV finali pronti per il DWH.
Il caricamento nel database avverrà via SQL con LOAD DATA INFILE oppure INSERT.
Oggi costruite quella pipeline. E costruite il Data Warehouse su cui girerà Power BI domani.

Agenda della giornata

Fascia	Attività
0:00–0:15	Brief del CTO.
0:15–3:30	Flowchart, schema DWH, script ETL, CSV di output, caricamento SQL.
3:30–4:00	Ogni team mostra il flowchart e spiega le scelte ETL.

Task G3

Task 3.1 — Flowchart ETL

Disegnate la pipeline prima di scrivere codice.

La pipeline deve mostrare:

- EXTRACT: da dove arrivano i dati?
- TRANSFORM: almeno 5 trasformazioni distinte.
- OUTPUT: che file produce lo script Python?
- LOAD: come vengono caricati nel DWH?
- Controlli qualità: dove si verifica la qualità dei dati?
- Gestione errori: cosa succede se qualcosa va storto?

Task 3.2 — Schema del Data Warehouse

Scrivete dwh_schema.sql.

```
CREATE DATABASE IF NOT EXISTS medistore_dwh;  
USE medistore_dwh;
```

```
CREATE TABLE dim_tempo (  
    id_tempo INT PRIMARY KEY AUTO_INCREMENT,  
    data DATE NOT NULL,  
    giorno TINYINT,  
    mese TINYINT,  
    nome_mese VARCHAR(20),  
    trimestre TINYINT,  
    anno YEAR,  
    giorno_settimana VARCHAR(15),  
    e_weekend BOOLEAN  
);
```

```
-- Completate voi le dim_prodotto, dim_farmacia, dim_cliente
```

```
CREATE TABLE fatto_vendite (  
    id_fatto INT PRIMARY KEY AUTO_INCREMENT,  
    id_tempo INT,  
    id_prodotto INT,  
    id_farmacia INT,  
    id_cliente INT,  
    quantita INT,  
    prezzo_unitario DECIMAL(10,2),  
    fatturato DECIMAL(10,2),  
    margine DECIMAL(10,2),  
    FOREIGN KEY (id_tempo) REFERENCES dim_tempo(id_tempo),  
    FOREIGN KEY (id_prodotto) REFERENCES dim_prodotto(id_prodotto),  
    FOREIGN KEY (id_farmacia) REFERENCES dim_farmacia(id_farmacia),
```

```
        FOREIGN KEY (id_cliente) REFERENCES dim_cliente(id_cliente)
    );
```

Task 3.3 — Script ETL

Scrivete `etl_pipeline.py`.

FASE EXTRACT

- Leggere i file `*_clean.csv`, `farmacie.csv`, `clienti.csv`, `fornitori_storico.json`.
- Stampare quanti record sono stati letti per ogni sorgente.

FASE TRANSFORM

...

Scrivere i record scartati in `scartati.csv` con il motivo.

FASE OUTPUT

- Scrivere i CSV finali per il DWH:
 - `dwh_dim_tempo.csv`
 - `dwh_dim_prodotto.csv`
 - `dwh_dim_farmacia.csv`
 - `dwh_dim_cliente.csv`
 - `dwh_fatto_vendite.csv`

REPORT FINALE (da stampare a schermo):

```
=== ETL PIPELINE REPORT ===
Record letti: ____
Record scartati: ____
Record duplicati: ____
Record caricati nel DWH: ____
=====
```

Task 3.4 — Caricamento nel DWH

Scrivete `carica_dwh.sql` che carica nel DWH i CSV prodotti dallo script ETL.

Usate `LOAD DATA LOCAL INFILE` oppure `INSERT`.

Ordine obbligatorio: dimensioni prima, fatto_vendite dopo.

Esempio:

```
USE medistore_dwh;
LOAD DATA LOCAL INFILE 'dwh_dim_tempo.csv'
INTO TABLE dim_tempo
FIELDS TERMINATED BY ',' ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
-- ripetuto per ogni tabella nell'ordine corretto
```

Task 3.5 — Data Quality Report

Compilete `data_quality_report.pdf` con:

- Elenco problemi trovati nella sorgente (tabella: file, colonna, problema, soluzione, n. record).
- Record scartati totali e motivazione principale.
- Valutazione qualità finale (1-10): completezza, accuratezza, consistenza.
- Raccomandazioni al cliente: cosa dovrebbe fare MediStore per migliorare la qualità alla fonte?

Deliverable Giornata 3

1. Flowchart ETL (foto o file)
2. `dwh_schema.sql`
3. `etl_pipeline.py` + output del report stampato
4. `carica_dwh.sql`
5. `scartati.csv`
6. `data_quality_report.md`

Aiuti disponibili G3

Gli aiuti si chiedono esplicitamente al docente. Ogni aiuto viene annotato.

Aiuto C1 – Generare `dim_tempo`

Per ogni data unica nelle vendite, estraete:

`giorno`, `mese`, `nome_mese`, `trimestre`, `anno`, `giorno_settimana`, `è_weekend`.

Usate la libreria standard `datetime` e il metodo `strftime`.

Aiuto C2 – Caricamento nel DWH

Python non scrive direttamente nel database in questa SAC.

Il vostro script produce CSV finali (dwh_*.csv).

Il file carica_dwh.sql li carica nel DWH con LOAD DATA LOCAL INFILE.

Aiuto C3 – Raccomandazioni al cliente

Non scrivete "noi puliremo i dati ogni mese".

Scrivete cosa deve cambiare alla fonte: formato data unico, campi obbligatori, validazione nel gestionale prima dell'esportazione.